

Codificação Estatística

Serviços de Rede e Aplicações Multimédia

Mestrado em Engenharia de Telecomunicações e Informática

Ivo Miguel Menezes Silva

2º Semestre (25/26)

Adaptação dos slides originais do Prof. Bruno Dias (Dept. Informática – UMinho)

CODIFICAÇÃO ESTATÍSTICA: Shannon-Fano (1948)

💡 IDEIA CENTRAL

Símbolos frequentes recebem códigos curtos
Símbolos raros recebem códigos longos

- ✓ Vantagem: Simples de entender e implementar
- ✗ Limitação: Em alguns casos não é tão eficiente como codificação de *Huffman* ou Aritmética

Shannon-Fano pode ser otimizado:

- Usar blocos de símbolos (em vez de 1 a 1) para aumentar a compressão
- Codificação/descodificação por matrizes binárias permite otimizar a velocidade computacional
- Suporta codificação adaptativa

Algoritmo de Codificação Shannon-Fano

1. Calcular as probabilidades P_i (ou as ocorrências) dos M símbolos do alfabeto. Criar uma tabela com uma linha para cada símbolo e atribuir o peso P_i a essa linha. Ordenam-se por ordem decrescente de peso. Inicia-se o algoritmo com um grupo com todos os símbolos e com todos os códigos como uma sequência vazia de bits.
2. Dividem-se as entradas consecutivas em dois novos grupos de forma a que a soma dos pesos dos grupos seja o mais aproximada possível.
3. Acrescentar o bit 0 aos códigos dos símbolos do grupo de cima e o bit 1 aos códigos dos símbolos do grupo de baixo.
4. Se algum dos dois grupos tiver apenas um símbolo, o código para esse símbolo está completo.
5. Se algum dos dois grupos tiver mais do que um símbolo, repete-se o algoritmo a partir do passo 2 para esse grupo.
6. Termina quando não sobrarem grupos com mais do que um símbolo.

Otimização de Codificação Shannon-Fano

É possível otimizar a codificação de Shannon-Fano

✓ Blocos Seletivos

Codificação com blocos seletivos permite otimizar a compressão sem ter de utilizar codificação com blocos para todos os símbolos. A decodificação é adaptada para verificar primeiro os prefixos maiores.

✓ Representação Canónica dos Códigos

Após a árvore final ser calculada, os códigos podem ser representados de forma mais compacta. Assim também não é preciso enviar a informação estatística ao descompressor para este calcular os códigos.

✓ Manipulação dos dados

A codificação da sequência de símbolos pode usar funções de manipulação de *arrays* de bytes sem necessidade de funções matemáticas mais lentas e de processamento de bits individuais.

Codificação Estatística: Huffman (1952)

💡 IDEIA CENTRAL

A codificação de *Huffman* comprime dados atribuindo códigos mais curtos aos símbolos mais frequentes e códigos mais longos aos menos frequentes.
Codificação criada por *D. Huffman* quando era aluno de *R. Fano*.

CARACTERÍSTICAS

- ✓ Calculam-se os códigos dos símbolos com as melhores probabilidades primeiro, normalmente utilizando árvores binárias (não precisa de recursividade).
- ✓ Obtêm-se códigos binários de **comprimento variável** tão bons ou melhores do que **Shannon-Fano**
- ✓ **Prefix-free** (livre de prefixos): significa que nenhum código binário para um símbolo é o prefixo (início) do código de qualquer outro símbolo. Isso garante uma decodificação única e inequívoca.
- ✓ Velocidade computacional pode ser otimizada com codificação/descodificação por matrizes binárias
- ✓ Suporta codificação adaptativa

Codificação Huffman (Clássico)

- Assume-se que as frequências (ou probabilidades) dos símbolos são conhecidas à partida (porque se faz a contagem de ocorrências no ficheiro, ou porque o modelo da fonte é conhecido).
- Constrói-se a árvore de Huffman uma vez, juntando iterativamente os dois nós de menor peso até obter a árvore final.
- **Codificador** e **descodificador** precisam de ter a mesma árvore (ou equivalentes: a tabela símbolo→código). Normalmente envia-se o cabeçalho com o alfabeto e comprimentos/códigos, ou assume-se um modelo conhecido.

Algoritmo de Codificação Huffman (Clássico)

1. Calcular as probabilidades P_i (ou as ocorrências) dos M símbolos do alfabeto. Criar uma árvore com uma única folha para cada símbolo e atribuir o peso P_i a essa folha. Inicia-se o algoritmo com M árvores.
2. Seja o peso duma árvore a soma dos pesos de todos os nós dessa árvore.
3. Criar uma nova árvore binária juntando as duas árvores com os pesos mais baixos. Fica-se assim com menos uma árvore.
4. Enquanto houver mais do uma árvore voltar ao passo 3.
5. Atribuir às ligações do lado esquerdo (entre um nó do nível superior e um nó do nível inferior) o bit 0 e às ligações do lado direito o bit 1.
6. O código de cada símbolo é a sequência de bits representada pelas ligações da árvore binária final, desde a raiz (nó de nível superior) até à folha representando esse símbolo.

Variações do Algoritmo de Codificação Huffman

A forma como as árvores são construídas (Como? Quando?) depende da variante de Huffman usada. O **codificador** e **descodificador** precisam de ter a mesma árvore, este processo pode ser feito tornando a árvore dinâmica (*Huffman adaptativo*) ou tornar os códigos mais bem estruturados (*Huffman canónico*).

Huffman Adaptativo

A árvore binária é construída e modificada dinamicamente conforme os símbolos vão sendo lidos. Os símbolos são codificados imediatamente (ou em pequenos grupos).

👍 **Vantagem:** Muito rápido — só lê o ficheiro uma única vez

👎 **Desvantagem:** Compressão ligeiramente inferior (a árvore não é global, está em construção)

👉 **Uso:** Ideal quando velocidade é crítica ou para dados em *streaming*.

Huffman Canónico

Modifica como os códigos são representados e armazenados após calcular a árvore completa. Recodifica-a numa forma mais compacta e regular.

👍 **Vantagens:** Codificação e descodificação computacionalmente simples e mais rápido, sem alterar a capacidade de compressão.

👉 **Uso:** *Standard* na prática porque é eficiente computacionalmente



EXEMPLO: Codificação Huffman

Codificação Huffman da sequência $S=\{ABABBCDCAA\}$. $P_A=0.4$; $P_B=0.3$; $P_C=0.2$; $P_D=0.1$.

✓ RESOLUÇÃO

Cálculo da árvore, passo a passo.

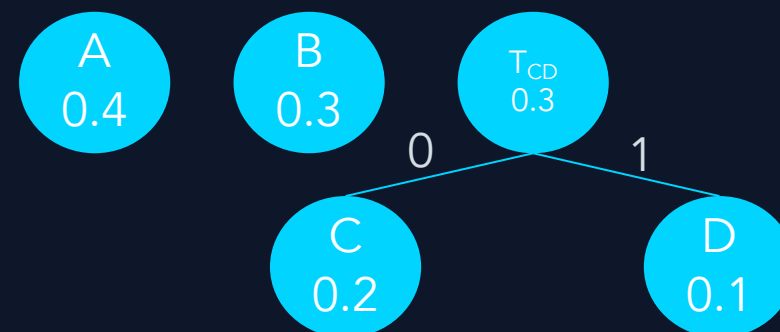
Inicialização: Começamos com alfabeto $a=\{A, B, C, D\}$, ordenando os símbolos pela sua frequência (probabilidade).

1º Passo: Criar uma *subtree* dos símbolos com frequências (probabilidades) mais baixas (C e D).

Inicialização



1º Passo





EXEMPLO: Codificação Huffman

Codificação Huffman da sequência $S=\{ABABBCDCAA\}$. $P_A=0.4$; $P_B=0.3$; $P_C=0.2$; $P_D=0.1$.

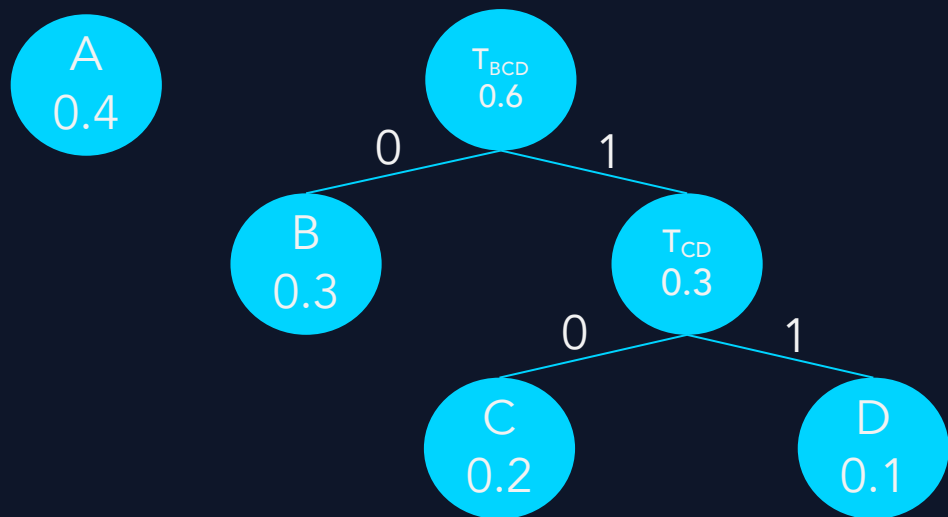
✓ RESOLUÇÃO

Cálculo da árvore, passo a passo.

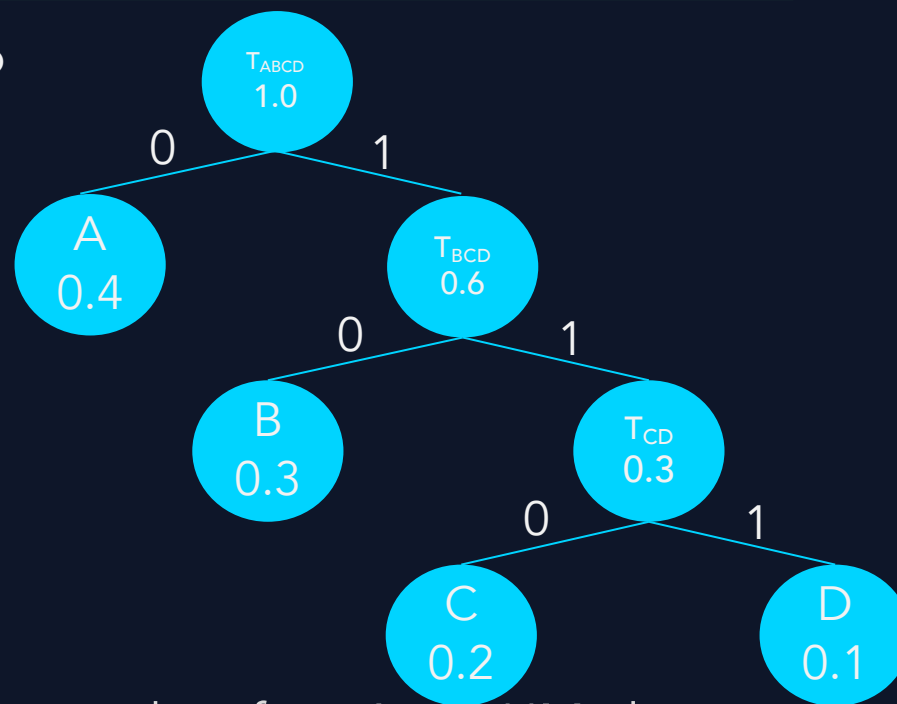
2º Passo: Criar nova *subtree* (T_{BCD}) dos símbolos de frequências mais baixas, ficamos com 2 árvores.

3º Passo: Criar árvore (T_{ABCD}) com todos os símbolos.

2º Passo



3º Passo





EXEMPLO: Codificação Huffman

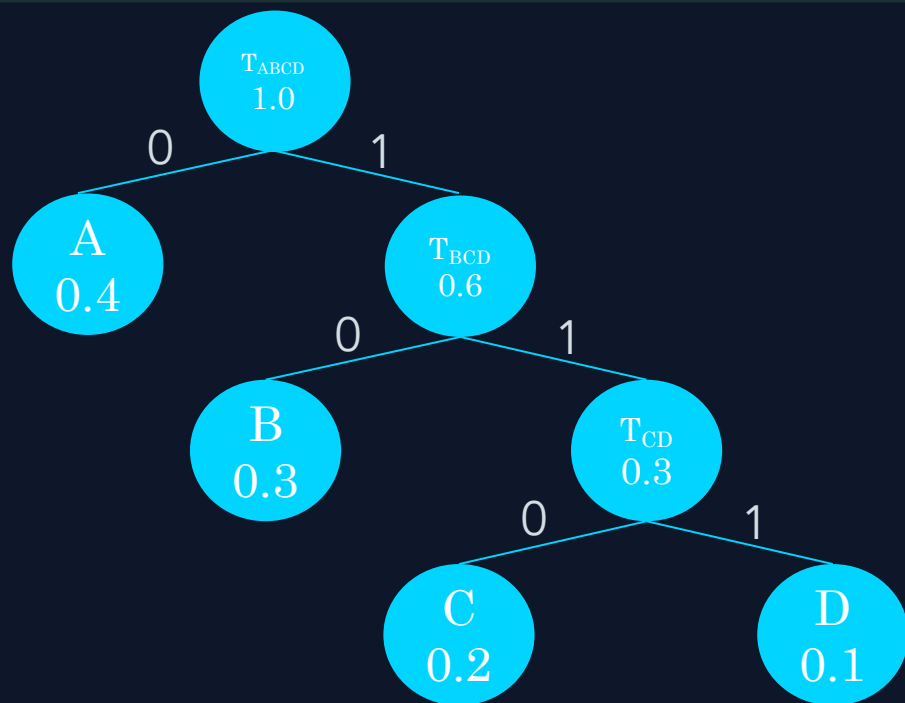
Codificação Huffman da sequência $S=\{ABABBCDCAA\}$. $P_A=0.4$; $P_B=0.3$; $P_C=0.2$; $P_D=0.1$.

✓ RESOLUÇÃO

Cálculo da árvore, passo a passo.

2º Passo: Criar nova *subtree* (T_{BCD}) dos símbolos de frequências mais baixas, ficamos com 2 árvores.

3º Passo: Criar árvore (T_{ABCD}) com todos os símbolos.



x_i	P_i	Código Huffman
A	0.4	0
B	0.3	10
C	0.2	110
D	0.1	111
Sequência a enviar $S=\{ABABBCDCAA\}$		0100101011011111000

Recursos auxiliares: Codificação Huffman

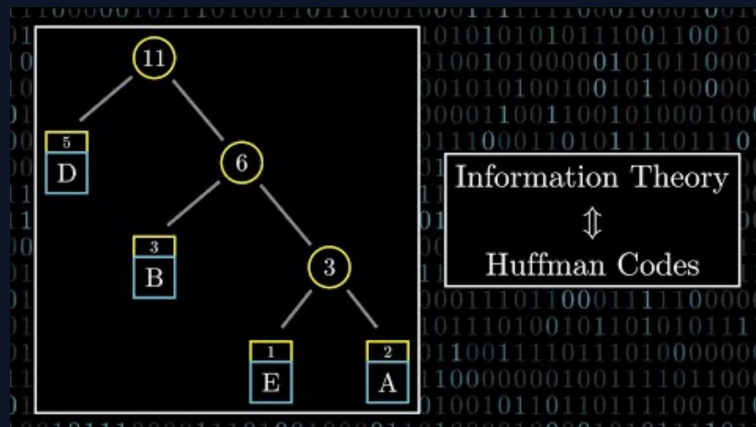
Huffman Compression Algorithm

BANANA BANDANA
001 000 010 000 010 000 101 001 000 010 100 000 010 000

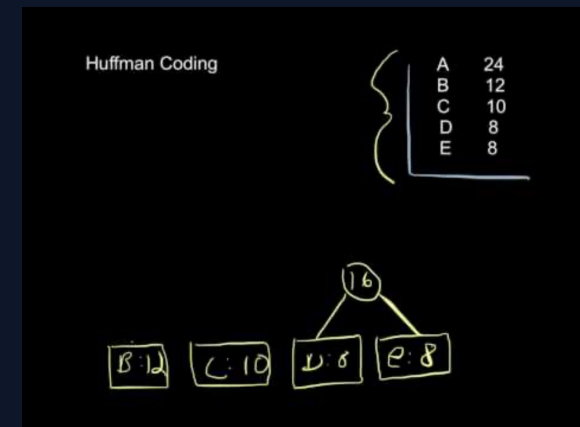
A : 000
B : 001
N : 010
D : 100

97 bits

<https://www.youtube.com/watch?v=0uCnBiy3Wuw>



<https://www.youtube.com/watch?v=B3y0RsVCyrw>



<https://www.youtube.com/watch?v=Ql18aET1G1w>

Representação Canónica dos Códigos

💡 DEFINIÇÃO

O **código canónico** mantém o mesmo comprimento de palavra de código para cada símbolo, mas renumera os códigos em ordem: primeiro pelo **comprimento**, depois pela **ordem alfabética** dos símbolos, atribuindo valores binários crescentes.

!! VANTAGENS

A principal vantagem da forma canónica é permitir a reconstrução de toda a tabela de códigos utilizando apenas o comprimento de cada símbolo, o que poupa memória e simplifica drasticamente o processo de descodificação.

- ✓ **Eficiência de armazenamento:** não é preciso guardar a estrutura da árvore de Huffman, apenas uma lista com o número de bits de cada letra.
- ✓ **Padronização:** como as regras de construção são fixas (ordem numérica e alfabética), o emissor e recetor chegam sempre ao mesmo código.
- ✓ **Velocidade:** facilita a implementação em sistemas reais, pois a descodificação pode ser feita através de tabelas de consulta rápidas, em vez de percorrer ramos de uma árvore.

Representação Canónica dos Códigos

Algoritmo:

1º Passo: Ordenar símbolos pelos comprimentos (código) crescentes (em caso de empate ordena-se pela ordem alfabética).

2º Passo: Atribuir códigos Canónicos:

- O primeiro símbolo recebe todos zeros com o seu comprimento.
- Cada símbolo seguinte recebe o próximo número binário (incrementar em 1) e, ao passar para um comprimento maior, faz-se "*shift*" à esquerda, acrescentando zeros até atingir o novo comprimento.

Exemplo de Representação Canónica dos Códigos

Obter a representação canónica do código considerando:
 $P(A)=0.5$; $P(B)=0.125$; $P(C)=0.125$; $P(D)=0.125$; $P(E)=0.125$

Symbol	Huffman Code	Length
A	0	1
E	100	3
D	101	3
C	110	3
B	111	3

EXERCÍCIO: Representação Canónica

Obter a representação canónica do código abaixo.

Symbol	Entropy Code	Len.
A	00	2
B	101	3
C	100	3
D	01	2
E	11111	5
F	110	3
G	1110	4
H	11110	5

Codificação Aritmética: Rissanen e R. Pasco (1976)



IDEIA CENTRAL

Não se calculam os códigos para os símbolos individuais mas calcula-se um número ou um intervalo (geralmente entre 0 e 1) que representa uma sequência completa de símbolos. Esse número é depois codificado, geralmente em binário.

CARACTERÍSTICAS

- Implementação computacional pode ser mais complexa do que Shannon-Fano ou Huffman
- Garante uma codificação teórica quase perfeita
- Suporta codificação adaptativa

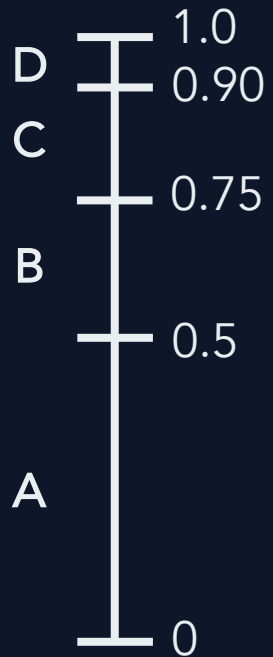
Algoritmo de Codificação Aritmética

1. Calculam-se as probabilidades P_i dos X_i símbolos da fonte, em que $P_1 \geq P_2 \geq \dots \geq P_M$ e $T = \sum_{i=1}^M P_i = 1$ (alfabeto de M símbolos $X_1 \dots X_M$).
2. Define-se o *range* inicial como $r = T$.
3. Seja $p_j = \sum_{i=1}^j P_i$ para $1 \leq j \leq M$ e p_0 o *offset*.
4. Inicia-se o número de símbolos codificados como $n = 0$ e $p_0 = 0$.
5. Para cada símbolo X_i ainda não processado da fonte
 - i. redefine-se o novo *offset* como $p_0 = p_{i-1} * r + p_0$,
 - ii. redefine-se o novo *range* como $r = P_i * r$ e, $n = n + 1$.
6. Se houver mais símbolos para codificar volta-se para o passo 5.
7. Se X_i for o último símbolo a codificar, o resultado numérico da codificação é o intervalo $[p_{i-1} * r + p_0, p_i * r + p_0]$ e o valor de n .
8. A sequência binária final é o número que possui menos bits e que representa um valor decimal (menor que 1) dentro do intervalo anterior.



EXEMPLO 1: Codificação Aritmética

Apresente a codificação aritmética para a sequência BACD, considerando $P(A)=0.5$, $P(B)=0.25$, $P(C)=0.15$, $P(D)=0.1$.



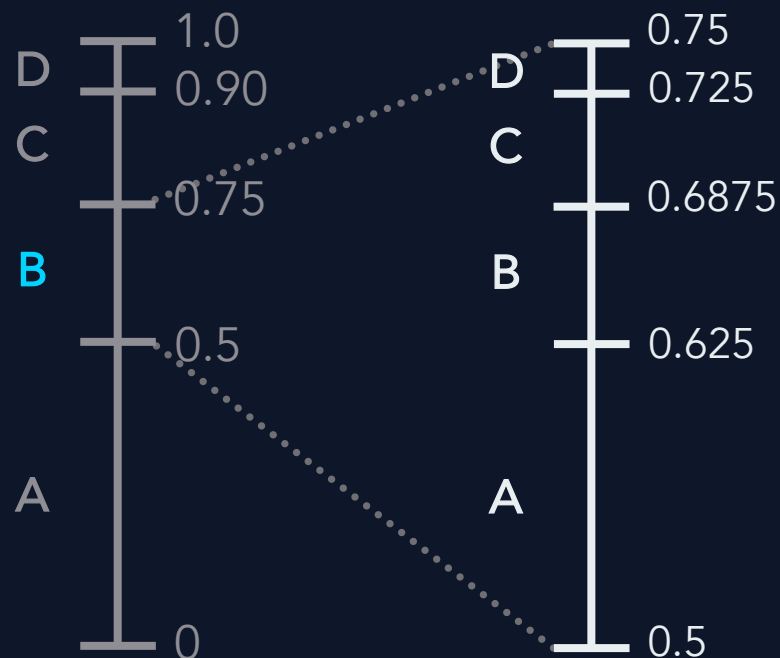
$A \in [0, 0.5)$
 $B \in [0.5, 0.75)$
 $C \in [0.75, 0.9)$
 $D \in [0.9, 1.0]$



EXEMPLO 1: Codificação Aritmética

Apresente a codificação aritmética para a sequência BACD, considerando:

$P(A)=0.5$, $P(B)=0.25$, $P(C)=0.15$, $P(D)=0.1$.



1º Passo: **Símbolo B**

Definir novo intervalo dado o símbolo B [0.5, 0.75):

$A_{LOW}=0.5$, $A_{HIGH}= A_{LOW}+0.25*P(A)=0.625$

$B_{LOW}=0.625$, $B_{HIGH}= B_{LOW}+0.25*P(B)=0.6875$

$C_{LOW}=0.6875$, $C_{HIGH}= C_{LOW}+0.25*P(C)=0.725$

$D_{LOW}=0.725$, $D_{HIGH}= D_{LOW}+0.25*P(D)=0.75$

Novo intervalo:

$A \in [0.5, 0.625)$

$B \in [0.625, 0.6875)$

$C \in [0.6875, 0.725)$

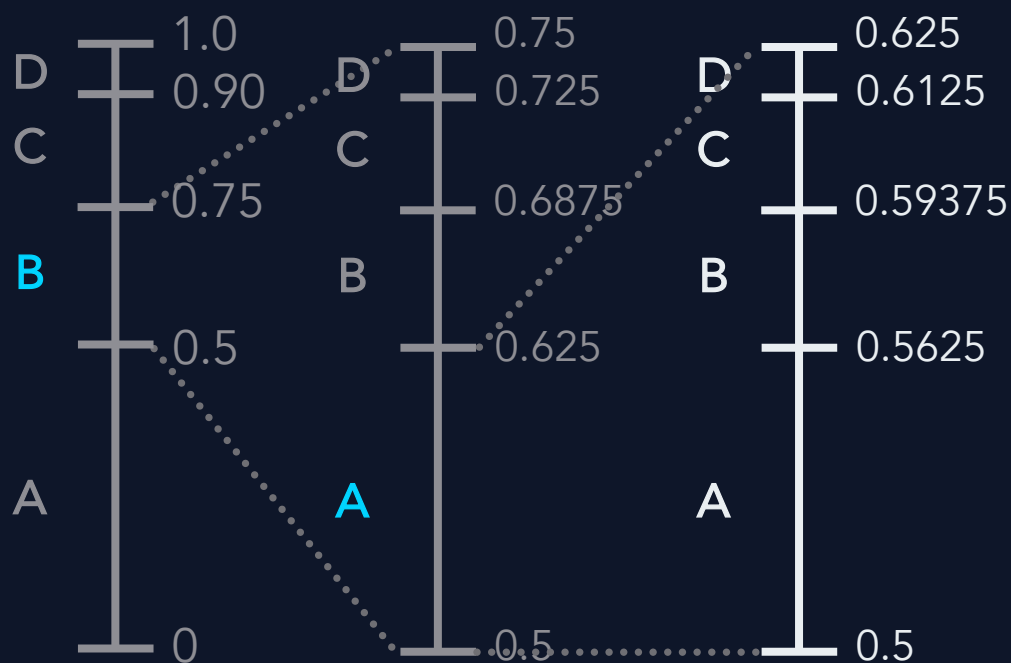
$D \in [0.725, 0.75)$



EXEMPLO 1: Codificação Aritmética

Apresente a codificação aritmética para a sequência BACD, considerando:

$P(A)=0.5$, $P(B)=0.25$, $P(C)=0.15$, $P(D)=0.1$.



2º Passo: **Símbolo A**

Definir novo intervalo dado o símbolo A [0.5, 0.625):

$A_{LOW}=0.5$, $A_{HIGH}= A_{LOW}+0.125*P(A)=0.5625$

$B_{LOW}= 0.5625$, $B_{HIGH}= B_{LOW}+0.125*P(B)=0.59375$

$C_{LOW}= 0.59375$, $C_{HIGH}= C_{LOW}+0.125*P(C)=0.6125$

$D_{LOW}=0.6125$, $D_{HIGH}= D_{LOW}+0.125*P(D)=0.625$

Novo intervalo:

$A \in [0.5, 0.5625)$

$B \in [0.5625, 0.59375)$

$C \in [0.59375, 0.6125)$

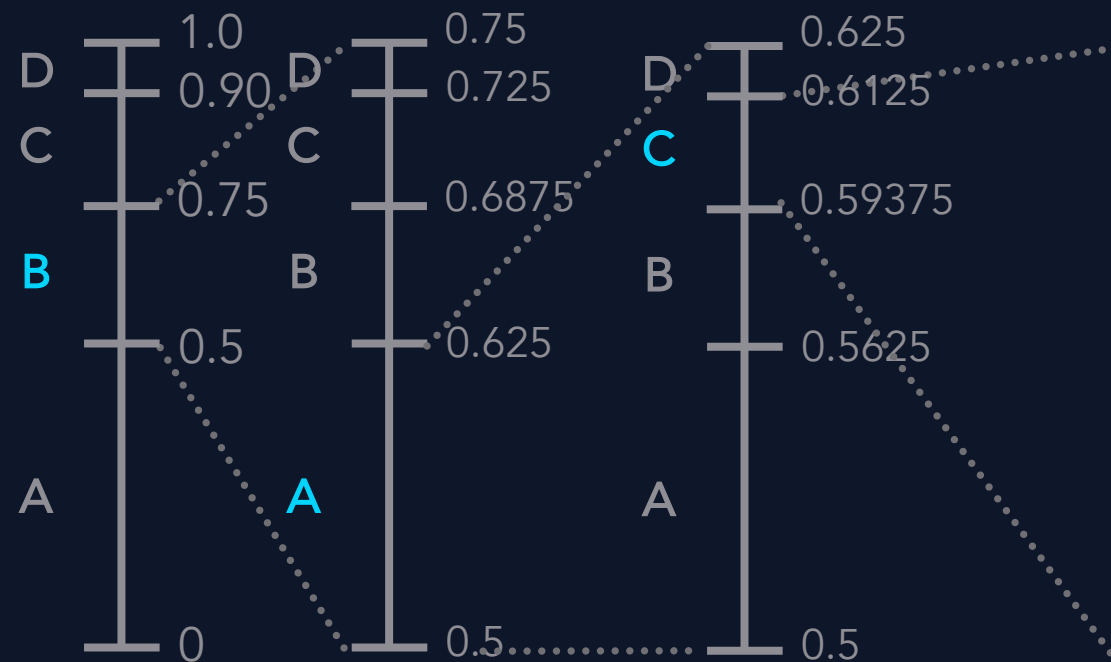
$D \in [0.6125, 0.625)$



EXEMPLO 1: Codificação Aritmética

Apresente a codificação aritmética para a sequência BACD, considerando:

$P(A)=0.5$, $P(B)=0.25$, $P(C)=0.15$, $P(D)=0.1$.



3º Passo: **Símbolo C**

Definir novo intervalo dado o símbolo C [0.59375, 0.6125):

$A_{LOW}=0.59375$, $A_{HIGH}= A_{LOW}+0.01875 \cdot P(A)=0.603125$

$B_{LOW}=0.603125$, $B_{HIGH}= B_{LOW}+0.01875 \cdot P(B)=0.6078125$

$C_{LOW}=0.6078125$, $C_{HIGH}= C_{LOW}+0.01875 \cdot P(C)=0.610625$

$D_{LOW}=0.610625$, $D_{HIGH}= D_{LOW}+0.01875 \cdot P(D)=0.6125$

Novo intervalo:

$A \in [0.59375, 0.603125)$

$B \in [0.603125, 0.6078125)$

$C \in [0.6078125, 0.610625)$

$D \in [0.610625, 0.6125)$

© iSilva 2026



EXEMPLO 1: Obtenção do Código Binário

Valor de cada bit em potência de 2

Bit m do código (0 ou 1)

Valor acumulado do código binário até ao bit m

Qualquer valor neste intervalo codifica a sequência completa. O objetivo é encontrar o **código binário mais curto** que caia dentro deste intervalo.

Número de bits testados

m	2^{-m}	b_m	$t = \sum_{i=1}^m 2^{-i} * b_i$	$0.610625 \leq t < 0.6125$
1	0.50000	1	0.50	<i>if</i> $b_1 = 1 \rightarrow t < 0.610625$
2	0.25000	0	0.50	<i>if</i> $b_2 = 1 \rightarrow t > 0.6125$

ALGORITMO

Para cada bit $m = 1, 2, 3, \dots$:

1. Calcula-se 2^{-m} (o "peso" desse bit)
2. Testa-se: Se somar este bit (fazer $b_m = 1$ resulta em $t + 2^{-m}$) ainda dentro do **intervalo**?
 - i. **SIM**: coloca-se $b_m = 1$ e soma-se $t \leftarrow t + 2^{-m}$:
 - ii. **NÃO**: coloca-se $b_m = 0$ e mantém-se t
3. Verifica-se se t já está **dentro** no intervalo (condição de paragem)



EXEMPLO 2: Codificação Aritmética

1º Passo

Codificação aritmética da sequência S $= \{AAABBCDAAA\}$					$X_1 = A$	$X_2 = B$	$X_3 = C$	$X_4 = D$
$P_A = 0,375; P_B = P_C = 0,25; P_D = 0,125$					$P_1 = 0,375$	$P_2 = 0,25$	$P_3 = 0,25$	$P_4 = 0,125$
		$n = 0$	$r = T = 1$	$p_0 = 0, p_{i-1} = 0$	$p_1 = 0,375$	$p_2 = 0,625$	$p_3 = 0,875$	$p_4 = 1$
S_n	i	$n = n + 1$	$r = P_i * r$	$p_0 = p_{i-1} * r + p_0$	$p_1 * r + p_0$	$p_2 * r + p_0$	$p_3 * r + p_0$	$p_4 * r + p_0$
A	1	1	1	[0	0,375[$//p_1^{(1)} = p_1 * r_1 + p_0^{(1)} = 0,375 * 1 + 0$	0,625 $//p_2^{(1)} = p_2 * r_1 + p_0^{(1)} = 0,625 * 1 + 0$	0,875 $//p_3^{(1)} = p_3 * r_1 + p_0^{(1)} = 0,875 * 1 + 0$	1 $//p_4^{(1)} = p_4 * r_1 + p_0^{(1)} = 1 * 1 + 0$

Símbolo da sequência

Nº Símbolos codificados

Índice do símbolo

Novo comprimento (range).
Range inicial $r=1$

Novo offset
(limite mínimo e limite máximo)

p_0 representa o limite inferior do intervalo atual. Este valor é copiado da iteração anterior.
Na primeira iteração é inicializado a zero



EXEMPLO 2: Codificação Aritmética

2º Passo

Codificação aritmética da sequência $S = \{AAABBCDAAA\}$					$X_1 = A$	$X_2 = B$	$X_3 = C$	$X_4 = D$
$P_A = 0,375$; $P_B = P_C = 0,25$; $P_D = 0,125$					$P_1 = 0,375$	$P_2 = 0,25$	$P_3 = 0,25$	$P_4 = 0,125$
		$n = 0$	$r = T = 1$	$p_0 = 0, p_{i-1} = 0$	$p_1 = 0,375$	$p_2 = 0,625$	$p_3 = 0,875$	$p_4 = 1$
S_n	i	$n = n + 1$	$r = P_i * r$	$p_0 = p_{i-1} * r + p_0$	$p_1 * r + p_0$	$p_2 * r + p_0$	$p_3 * r + p_0$	$p_4 * r + p_0$
A	1	1	1	[0	0,375[//0,375 * 1 + 0	0,625 //0,625 * 1 + 0	0,875 //0,875 * 1 + 0	1 //1 * 1 + 0
A	1	2 //n = 1 + 1	0,3750 //r ₂ = P _i * r ₁ = 0.3750 * 1	[0	0,1406250000000[//0,375 * 0,375 + 0	0,2343750000000 //0,625 * 0,375 + 0	0,3281250000000 //0,875 * 0,375 + 0	0,3750000000000 //1 * 0,375 + 0

Símbolo da sequência

Nº Símbolos codificados

Índice do símbolo

Novo comprimento (range).

Novo offset (limite mínimo e limite máximo)



EXEMPLO 2: Codificação Aritmética

10º Passo – Codificação concluída

Codificação aritmética da sequência $S = \{AAABBCDAAA\}$

$P_A = 0,375$; $P_B = P_C = 0,25$; $P_D = 0,125$

$X_1 = A$

$X_2 = B$

$X_3 = C$

$X_4 = D$

$P_1 = 0,375$

$P_2 = 0,25$

$P_3 = 0,25$

$P_4 = 0,125$

$n = 0$

$r = T = 1$

$p_0 = 0, p_{i-1} = 0$

$p_1 = 0,375$

$p_2 = 0,625$

$p_3 = 0,875$

$p_4 = 1$

S_n	i	$n = n + 1$	$r = P_i * r$	$p_0 = p_{i-1} * r + p_0$	$p_1 * r + p_0$	$p_2 * r + p_0$	$p_3 * r + p_0$	$p_4 * r + p_0$
A	1	1	1	[0	0,375[	0,625	0,875	1
A	1	2	0,3750000000000000	[0	0,14062500000000[	0,23437500000000	0,32812500000000	0,37500000000000
A	1	3	0,1406250000000000	[0	0,05273437500000[	0,08789062500000	0,12304687500000	0,14062500000000
B	2	4	0,0527343750000000	0	[0,0197753906250	0,0329589843750[	0,0461425781250	0,05273437500000
B	2	5	0,0131835937500000	0,01977539062500	[0,0247192382813	0,0280151367188[	0,0313110351563	0,0329589843750
C	3	6	0,003295898437500	0,02471923828125	[0,0259552001953	[0,0267791748047	0,0276031494141[	0,0280151367188
D	4	7	0,000823974609375	0,02677917480469	0,0270881652832	[0,0272941589355	[0,0275001525879	0,0276031494141[
A	1	8	0,000102996826172	[0,02750015258789	0,0275387763977[	0,0275645256042	0,0275902748108	0,0276031494141
A	1	9	0,000038623809814	[0,02750015258789	0,0275146365166[	0,0275242924690	0,0275339484215	0,0275387763977
A	1	10 //n = 9 + 1	0,000014483928680 //r ₁₀ = r ₉ * P ₁ = 0,000038623809814 * 0,375	[0,02750015258789 //mantém-se da iteração anterior	0,0275055840611[//p ₁ ⁽¹⁰⁾ = p ₁ * r ₁₀ + p ₀ ⁽¹⁰⁾ = 0,375 * 0,000014483928680 + 0,02750015258789	0,0275092050433 //p ₂ ⁽¹⁰⁾ = p ₂ * r ₁₀ + p ₀ ⁽¹⁰⁾	0,0275128260255 //p ₃ ⁽¹⁰⁾ = p ₃ * r ₁₀ + p ₀ ⁽¹⁰⁾	0,0275146365166 //p ₄ ⁽¹⁰⁾ = p ₄ * r ₁₀ + p ₀ ⁽¹⁰⁾



EXEMPLO 2: Obtenção do Código Binário

Valor de cada bit em potência de 2

Bit m do código (0 ou 1)

Valor acumulado do código binário até ao bit m

Qualquer valor neste intervalo codifica a sequência completa. O objetivo é encontrar o **código binário mais curto** que caia dentro deste intervalo.

Número de bits testados

m	2^{-m}	b_m	$t = \sum_{i=1}^m 2^i * b_i$	$0,02750015258789 \leq t < 0,0275055840611$
1	0,5000000000000000	0	0,0000000000000000	$if\ b_1 = 1 \rightarrow t > 0,0275055840611$
2	0,2500000000000000	0	0,0000000000000000	$if\ b_2 = 1 \rightarrow t > 0,0275055840611$

ALGORITMO

Para cada bit $m = 1, 2, 3, \dots$:

1. Calcula-se 2^{-m} (o "peso" desse bit)
2. Testa-se: Se somar este bit (fazer $b_m = 1$ resulta em $t + 2^{-m}$) ainda dentro do **intervalo**?
 - i. **SIM**: coloca-se $b_m = 1$ e soma-se $t \leftarrow t + 2^{-m}$:
 - ii. **NÃO**: coloca-se $b_m = 0$ e mantém-se t
3. Verifica-se se t já está **dentro** no intervalo (condição de paragem)



EXEMPLO 2: Obtenção do Código Binário

m	2^{-m}	b_m	$t = \sum_{i=1}^m 2^{-i} * b_i$	$0,02750015258789 \leq t < 0,0275055840611$
1	0,5000000000000000	0	0,0000000000000000	$\text{if } b_1 = 1 \rightarrow t > 0,0275055840611$
2	0,2500000000000000	0	0,0000000000000000	$\text{if } b_2 = 1 \rightarrow t > 0,0275055840611$
3	0,1250000000000000	0	0,0000000000000000	$\text{if } b_3 = 1 \rightarrow t > 0,0275055840611$
4	0,0625000000000000	0	0,0000000000000000	$\text{if } b_4 = 1 \rightarrow t > 0,0275055840611$
5	0,0312500000000000	0	0,0000000000000000	$\text{if } b_5 = 1 \rightarrow t > 0,0275055840611$
6	0,0156250000000000	1	0,0156250000000000	$b_6 = 1 \rightarrow t < 0,02750015258789$
7	0,0078125000000000	1	0,0234375000000000	$b_7 = 1 \rightarrow t < 0,02750015258789$
8	0,0039062500000000	1	0,0273437500000000	$b_8 = 1 \rightarrow t < 0,02750015258789$
9	0,0019531250000000	0	0,0273437500000000	$\text{if } b_9 = 1 \rightarrow t > 0,0275055840611$
10	0,0009765625000000	0	0,0273437500000000	$\text{if } b_{10} = 1 \rightarrow t > 0,0275055840611$
11	0,0004882812500000	0	0,0273437500000000	$\text{if } b_{11} = 1 \rightarrow t > 0,0275055840611$
12	0,0002441406250000	0	0,0273437500000000	$\text{if } b_{12} = 1 \rightarrow t > 0,0275055840611$
13	0,0001220703125000	1	0,0274658203125000	$b_{13} = 1 \rightarrow t < 0,02750015258789$
14	0,0000610351562500	0	0,0274658203125000	$\text{if } b_{14} = 1 \rightarrow t > 0,0275055840611$
15	0,0000305175781250	1	0,0274963378906250	$b_{15} = 1 \rightarrow t < 0,02750015258789$
16	0,0000152587890625	0	0,0274963378906250	$\text{if } b_{16} = 1 \rightarrow t > 0,0275055840611$
17	0,0000076293945313	1	0,0275039672851562	✅ condição verificada
Código binário $\{b_1 b_2 b_3 \dots b_m\} = \mathbf{00000111000010101}$				

Outras famílias de Codificação por Entropia

- A codificação aritmética pura tem **problemas de implementação computacional** (na representação dos valores dos limites do intervalo final e na construção da sequência binária que tem um valor decimal dentro do intervalo definido). Além disso, a parte matemática do algoritmo, embora conceitualmente simples, é computacionalmente pouco eficiente de implementar diretamente.
- Por estas razões apareceram mecanismos baseados no mesmo conceito mas **extremamente eficientes computacionalmente**, ainda que não tenham uma capacidade tão grande de otimização da compressão. A família de algoritmos mais conhecida e utilizada é a dos Sistemas Numéricos Assimétricos. Este grupo de algoritmos está normalizado no RFC8878 e é muito utilizado para compressão em sistemas operativos (Linux e Android), aplicações (Facebook, Dropbox, etc.) e embebidos noutros mecanismos de compressão (para MIME e HTTP, no Apple LZFS, no Google Draco 3D, no JPEG XL, etc.).